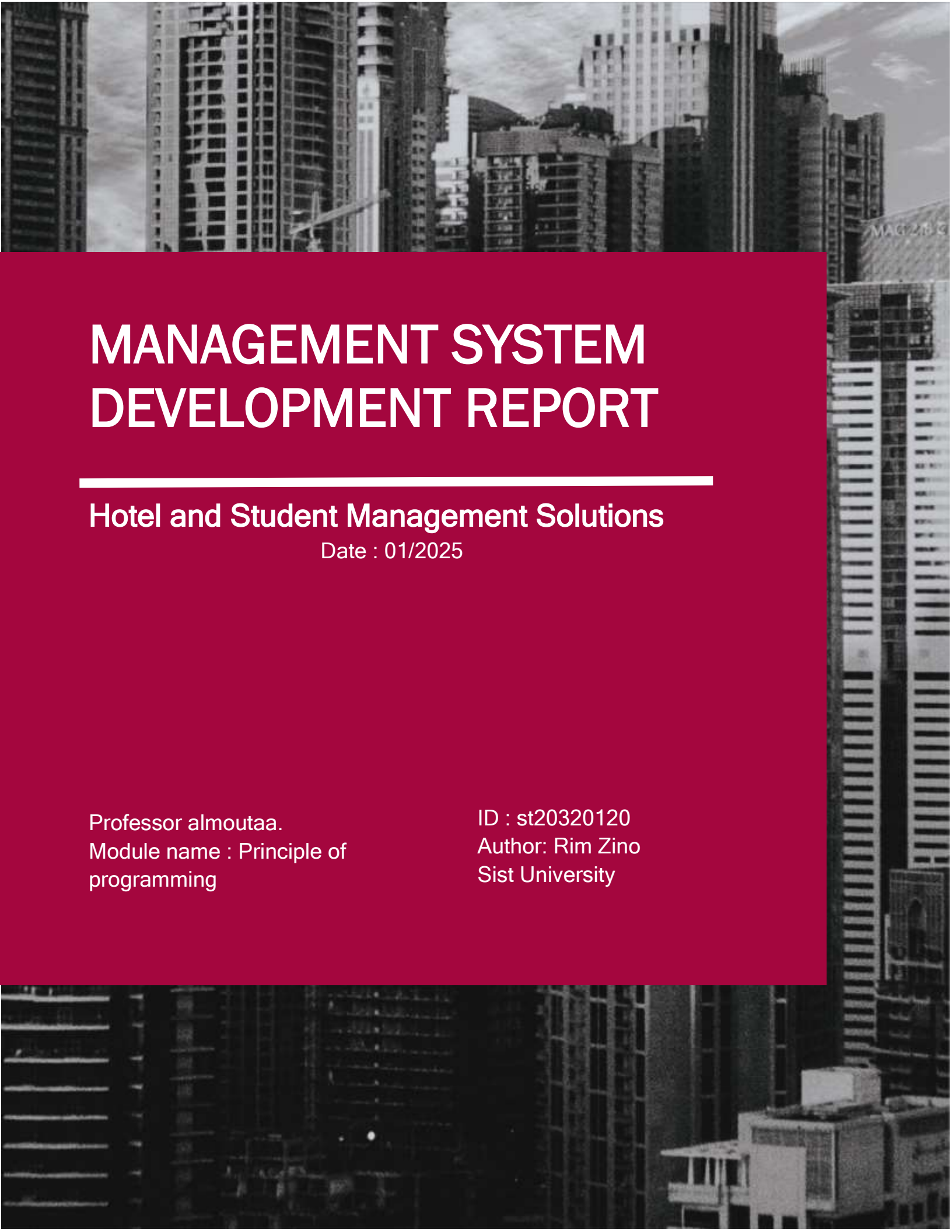


MANAGEMENT SYSTEM DEVELOPMENT REPORT

Hotel and Student Management Solutions

Date : 01/2025

Professor almoutaa.
Module name : Principle of programming

ID : st20320120
Author: Rim Zino
Sist University

Contents

the working system:..... 5
Costumer's Data: 6
Room rent: 6
Restaurant bill: 7
Laundry bill: 10
Games bill: 11
Total cost: 12
The Working system: 16
The List: 17
Add student: 19
View student: 19
Search student: 20
Remove student: 20

INTRODUCTION

- Management systems are basically tools and methodologies that help an organization to achieve efficiency and effectiveness in its operations. The systems allow organizations to plan, organize, and monitor their activities toward the achievement of set goals in a well-organized manner. In this report, I will be presenting two different management systems: a hotel management system and a student management system. Both systems were implemented in Java because it is one of the most versatile and efficient languages for the development of projects. The adaptability and strength of Java make it one of the best choices for creating scalable applications that would meet user needs on a continuing basis.

- The first system is a hotel management system, which is an easy-operated system and does not need to include the log-in feature. The system handles customer data, calculates the cost of room rental, processes the restaurant bills, handles the laundry services, accounts for game activities, and computes the total cost for the customer. The system addresses challenges associated with traditional manual booking processes, which often take time and lead to human error. Ogirima and Omuya (n.d.) point out that such systems represent effective tools in providing speed and safety while recording customer information, but at the same time, they help and make the life of the hotel staff easier since it is possible to monitor customer activities and expenses while reducing the possibility of certain mistakes.

- The hotel management system enhances the customer's satisfaction by automating these processes and improving the efficiency of the overall running of the hotel.

- On the other hand, The Student Management System is an administrative application designed for effective management of student records. It provides the user with the facility to insert, view, search, and delete student information. There is no login feature required in this system.

- It ensures that student information is accessible, secure, and organized. The student management system helps the administrative staff in keeping each record of students systematically and in an organized manner by reducing the fallacies of manual traditional systems. Its simplicity in interface helps to make it user-friendly even for relatively less techno-savvy people, adding to its practicality and usefulness. This report will explain the architecture, functionality, and deployment of the two systems. In the context of the hotel management system, it would be explained how the system was collecting data from the customers on one platform and was calculating all sorts of different bills, to sum up in the end how everything is compiled over to one place to prove that it could compile most aspects of running a hotel onto one single platform. Similarly, the student management system will be analyzed in order to show if it can cater to critical administrative tasks like additions, viewing, and management within the student database. Attention to both systems shall be given towards their respective complementary features that result in improved working and usability from both systems. This report, therefore, undertakes a comparative analysis in depth into the design and implementation of these two systems, showing how Java-based applications can be leveraged to create robust and efficient management solutions that address organizational needs.

Hotel Management System

the working system:

- The hotel Management System work in a very simple way where the user enters his/her information including the name, address, check-in and check-out dates, then it comes next to choose which room type desired as I mentioned 4 rooms type: Single, Double, Deluxe and suite. The user should select a room to book and add it to the system.
- When we say a hotel, we say also a restaurant and for that I made a menu include some desired dishes that could be available in the restaurant if not then it appears as an invalid choice, else when they select the desired dishes the system counts the total cost.
- In addition, I made a laundry menu with prices on each element, the user should select as many elements as possible, and the system prints the total price. The next to come is the game's part, where there is a small menu of available tickets games where the user should choose one of the tickets and select how many hours to spend, as a result the system will print the total price.
- In the end the system will print the final total price and costumer details including the costumer's data, room, restaurant, laundry and games bill.

THE CODE

Costumer's Data:

- First, in the code process I started with importing the scanner as I'm going to use the scanner in my code for the input, then I opened a class named "Hotel Aurora".
- After these steps I moved to the next one where the costumer should enter the information, I printed the words that shows what the user should do for example "Enter check-in date:", after that I put a scanner where the user should write the date or the answer.

```
import java.util.Scanner;

public class HotelAurora {
    Run | Debug
    public static void main(String[] args) {
        //  Enter Costumer's data
        Scanner scan = new Scanner(System.in);
        System.out.println(x:"Welcome to Hotel Aurora");
        System.out.println(x:"Enter Full Name :");
        String name = scan.nextLine();
        System.out.println(x:"Enter Adress :");
        String adress = scan.nextLine();
        System.out.println(x:"Enter Check-in Date :");
        String checkindate = scan.nextLine();
        System.out.println(x:"Enter Check-out Date :");
        String checkoutdate = scan.nextLine();
```

Room rent:

- In this part I made a room type selection, where I printed 4 room types and put a scanner for the user to select the desired room type. The next step I printer the number of nights stayed also with a scanner for the answer. And finally combining them by making a calculation with the control statement "switch". According to the *Java Language Specification (Oracle, 2015)*, "The switch statement transfers control to one of several

statements depending on the value of an expression." And this provides me with the desired results, in the end the user enters the choice of room type and how many nights then it comes the calculation role where it calculates the price per night.

```
// Rooms type selection
System.out.println(x:" Rooms type ");
System.out.println(x:"1. Single Room - 100$ per night");
System.out.println(x:"2. Double Room - 150$ per night");
System.out.println(x:"3. Deluxe Room - 250$ per night");
System.out.println(x:"4. Suite Room - 300$ per night");
System.out.println(x:"Selecte Room type :");
int roomtype = scan.nextInt();

// Room type calculation
System.out.println(x:"Enter number of nights :");
int nights = scan.nextInt();
double roomrent = 0;
switch (roomtype) {
    case 1: roomrent = 100 * nights;break;
    case 2: roomrent = 150 * nights;break;
    case 3: roomrent = 250 * nights;break;
    case 4: roomrent = 300 * nights;break;
    default: roomrent = 0;
    System.out.println(x:"Invalid Room type selected.");
        break;
}
System.out.println("Total room price : $"+roomrent);
```

Restaurant bill:

- In every hotel there is a restaurant, and, in every restaurant, there is a menu. In this part of the code, I made a menu list and printed it. It comes with a scanner where to put the answer, and we don't forget the switch statement to choose one choice out of a lot of choices depending on the values selected. However, if we run it the output will provide us to choose just one choice, in this case we had to make a loop, the while loop.

  Since the role of the while loop according to *Oracle Corporation, 2014,* "The while statement continually executes a block of statements while a particular condition is true.", and inside of this loop I made an if loop.

According to *Rebus Press (n.d.),* the if loop has a big role in decision-making within programs. And for that I made if the food item equal to 0 means move to the next step, to make it clearer the user will enter as many choices as desired and if the user had made enough choices, he/she must enter 0 to submit the choices and calculate the total price.

```
// Display menu items and read user's choice.
double restaurantbill = 0;
int fooditem;
while (true) {

    System.out.println(x:"  Restaurant menu  ");
    System.out.println(x:" Appetizers :");
    System.out.println(x:"1. Bruschetta - $8");
    System.out.println(x:"2. Caesar Salad - $12");
    System.out.println(x:"3. Spring Rolls (Vegetarian) - $10");
    System.out.println(x:" Main courses :");
    System.out.println(x:"4. Grilled Chicken Breast - $18");
    System.out.println(x:"5. Beef Tenderloin Steak - $28");
    System.out.println(x:"6. Vegetarian Lasagna - $16");
    System.out.println(x:"7. Fish and Chips - $20");
    System.out.println(x:" Specialities :");
    System.out.println(x:"8. Lobster Tail - $35");
    System.out.println(x:"9. Seafood Paella - $30");
    System.out.println(x:"10. Chicken Parmesan - $22");
    System.out.println(x:"Side dishes :");
    System.out.println(x:"11. French Fries - $5");
    System.out.println(x:"12. Steamed Vegetables - $6");
    System.out.println(x:"13. Garlic Mashed Potatoes - $7");
    System.out.println(x:"14. Rice Pilaf - $5");
    System.out.println(x:" Desserts :");
    System.out.println(x:"15. Cheesecake - $8");
    System.out.println(x:"16. Chocolate Lava Cake - $9");
    System.out.println(x:"17. Tiramisu - $7");
    System.out.println(x:" Beverages :");
```

```
System.out.println(x:"18. Soft Drinks - $3");
System.out.println(x:"19. Fresh Juice (Orange, Apple, Cranberry) - $5");
System.out.println(x:"20. Coffee or Tea - $4");
System.out.println(x:"21. Wine (Glass) - $10");
System.out.println(x:"22. Beer (Bottle) - $6");
System.out.println(x:"0. Next.");
System.out.println(x:"Select food item (enter 0 to Move Next):");
fooditem = scan.nextInt();

// loop if 0 selected move next
if (fooditem == 0) {
    break;
}


    // Adding prices.
switch (fooditem) {
    case 1: restaurantbill += 8 ;break;
    case 2: restaurantbill += 12;break;
    case 3: restaurantbill += 10 ;break;
    case 4: restaurantbill += 18 ;break;
    case 5: restaurantbill += 28 ;break;
    case 6: restaurantbill += 16 ;break;
    case 7: restaurantbill += 20 ;break;
    case 8: restaurantbill += 35 ;break;
    case 9: restaurantbill += 30 ;break;
    case 10: restaurantbill += 22 ;break;
    case 11: restaurantbill += 5 ;break;
    case 12: restaurantbill += 6 ;break;
    case 13: restaurantbill += 7 ;break;
```

```
    case 13: restaurantbill += 7 ;break;
    case 14: restaurantbill += 5 ;break;
    case 15: restaurantbill += 8 ;break;
    case 16: restaurantbill += 9 ;break;
    case 17: restaurantbill += 7 ;break;
    case 18: restaurantbill += 3 ;break;
    case 19: restaurantbill += 5 ;break;
    case 20: restaurantbill += 4 ;break;
    case 21: restaurantbill += 10;break;
    case 22: restaurantbill += 6 ;break;
    default:
        System.out.println(x:" Invalid choice! please try again.");
        break;
}

System.out.println("food price :$"+restaurantbill); // Print food price selected
```

Laundry bill:

I made a list through the use of print and scan statements so that it can be viewed by, and input from, the user. Then, through the use of a switch statement, I assigned prices according to the choices. A loop was used so that multiple selections may be chosen for flexibility and ease of the user. Lastly, the total price was determined through the summation of all items selected.

```
// Display laundry items and read user's selection.
double laundryprice = 0;
int laundry;
while (true) {

    System.out.println(x:" Laundry prices");
    System.out.println(x:"1. Shirt/Blouse - 5$");
    System.out.println(x:"2. Pants/Trousers - 7$");
    System.out.println(x:"3. Dress - 10$");
    System.out.println(x:"4. Suit(jacket + pants) - 15$");
    System.out.println(x:"5. Towel - 3$");
    System.out.println(x:"selected item or (enter 0 to Move Next) :");
    laundry = scan.nextInt();

    if (laundry == 0) {
        break;
    }


    // Adding prices
    switch (laundry) {
        case 1: laundryprice += 5;break;
        case 2: laundryprice += 7;break;
        case 3: laundryprice += 10;break;
        case 4: laundryprice += 15;break;
        case 5: laundryprice += 3;break;
        default:
            System.out.println(x:"Invalid choice! plese try again.");
            break;
    }
```

```
System.out.println("Total laundry cost: $" + laundryprice); // print total price selected
```

Games bill:

- Now, in this part, with the help of program design, I provide a menu of selecting the type of ticket by showing different tickets available to him. This would be done simply by printing on a console some available ticket types that he could choose and see accordingly. For making the choice regarding the selection he provides, I then used a scan object to capture the choice on the console which is already selected.
- After the selection of the ticket, the program goes to the second stage, which is to select how many hours he or she wants to play. The user will input the number of hours, which later will be used for price calculation. For the above inputs, I implemented switch statements to map the types of tickets to the respective prices. This structure will make sure each ticket choice is processed in accordance with some predefined pricing logic.
- It then calculates the total price of the ticket, being the price per hour of the type of ticket chosen times the number of hours that the user has input. It prints out the price calculated as the total cost, therefore providing clarity to the user on the output of the total cost for the user-chosen activities. This is, therefore, a design to keep the user interactive while ensuring simplicity and clarity on how the determination of the total price

for user-selected activities will be made.

```
// Display games tickets type and read user's selection.
System.out.println(x:" Games Tickets");
System.out.println(x:"1. Board Games Ticket - 3$ per hour");
System.out.println(x:"2. Outdoor Games Ticket - 7$ per hour");
System.out.println(x:"3. Virtual Reality Ticket - 50$ per hour");
System.out.println(x:"Select a ticket :");
int tickettype = scan.nextInt();

System.out.println(x:"Enter number of hours :");
int hours = scan.nextInt();
double tickets = 0;

// calculate prices per hours
switch (tickettype) {
    case 1: tickets = 3 * hours;break;
    case 2: tickets = 7 * hours;break;
    case 3: tickets = 50 * hours;break;
    default: tickets = 0;
        System.out.println(x:" Invalid Ticket selected.");
        break;
}
System.out.println("Ticket Price per hours :$"+tickets); // print final price.
```

Total cost:

- This final part finally assembles all of the above: customer information, food bill at a restaurant, laundry, and game bill. In the end, it calculates the total amount price for the summation purpose of customer data. In this

way, there will not be a single dollar not accounted for.

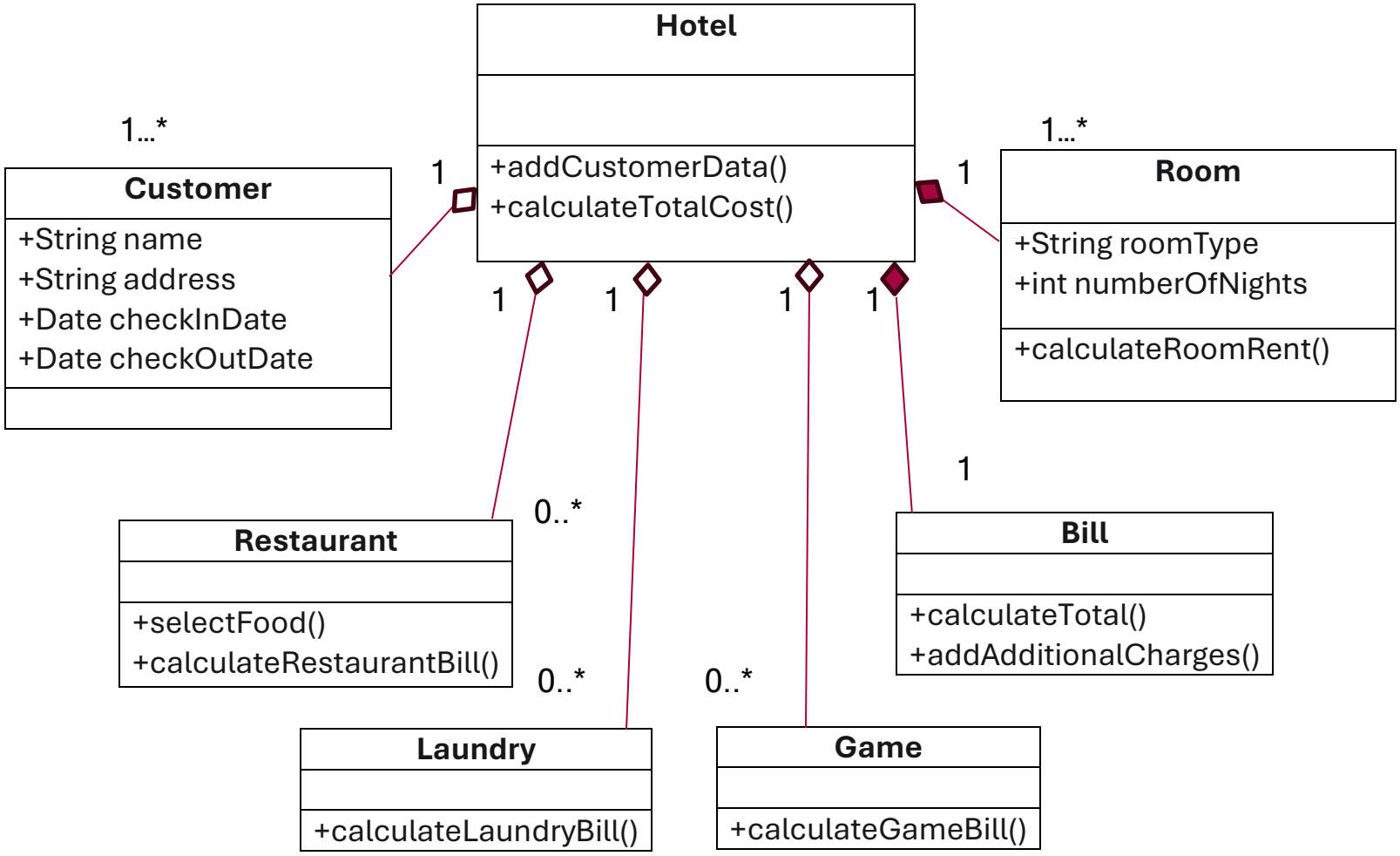
```
// Display summary of costumer's detail information.
System.out.println("Full name :"+name);
System.out.println("Adress :"+adress);
System.out.println("Check-in Date :"+checkindate);
System.out.println("Check-out Date :"+checkoutdate);
System.out.println("Room Type :"+roomtype);
System.out.println("Room Price :$"+roomrent);
System.out.println("Restaurant bill :$"+restaurantbill);
System.out.println("Laundry bill :$"+laundryprice);
System.out.println("Games Ticket cost :$"+tickets);

double totalCost = roomrent + restaurantbill + laundryprice + tickets;
System.out.println("Total costs: $" + totalCost); // print total cost.

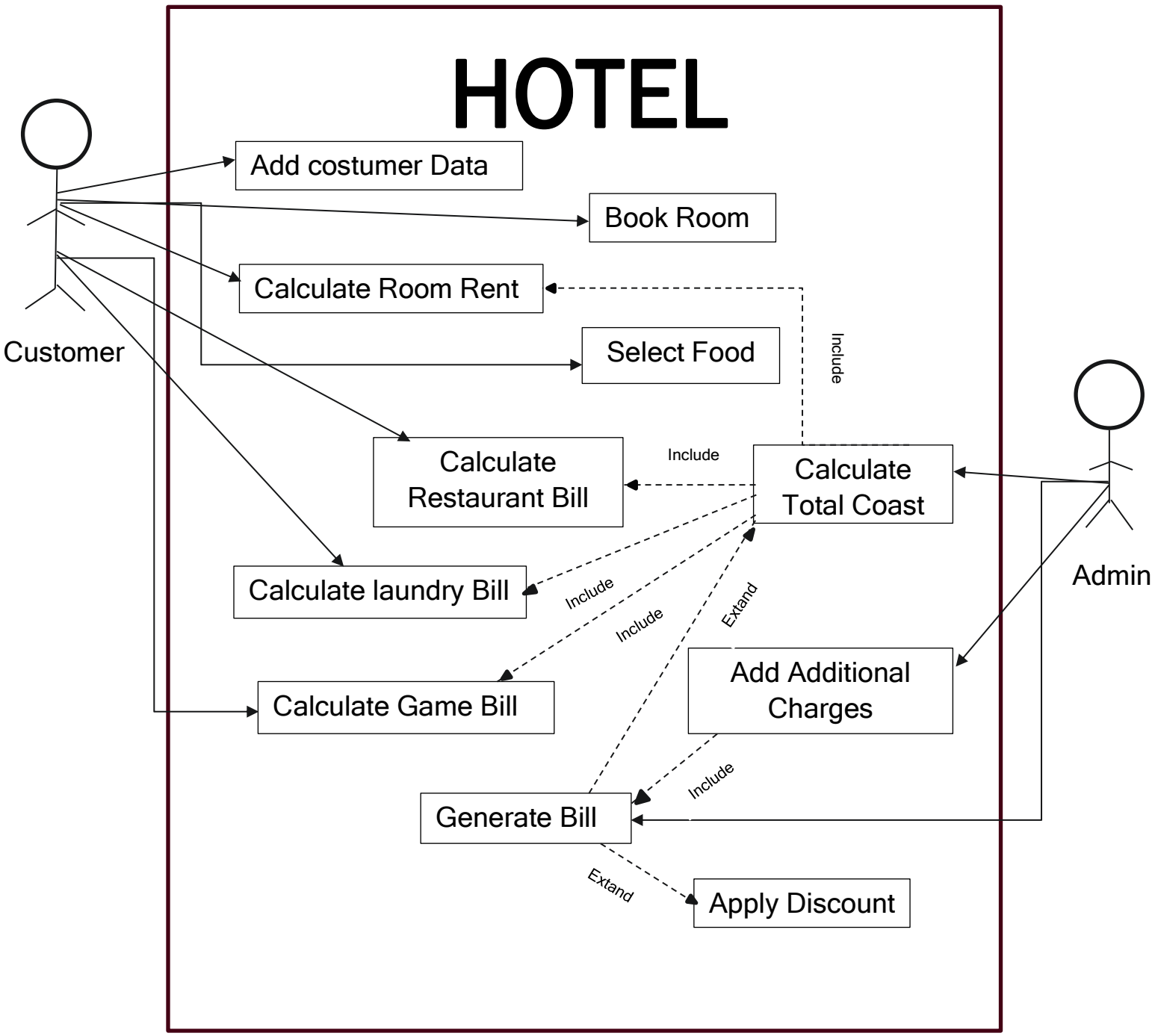
scan.close();
```

UML MODELING

- According to *Booch, Rumbaugh, and Jacobson (1999),* UML is a system of visual representation which is hierarchical and structured that allows computing professionals to document and build software artifacts as well as visualize them. This aids in the development of models for software systems in different forms, making communication of ideas around the system, and the designing of the system itself, easier.
- And for the hotel management system we can use a UML diagram:



USE CASE



This is the use case diagram for Hotel Management System, comprising three main actors: the Customer, the Admin, and the system itself. The customer has the privilege to perform the following actions: adding the customer data, booking the rooms, choosing the food in the restaurant, and calculating the bill to be played. Admin is responsible for managing the work of adding extra charges to the bill, generating the bill, and giving discounts. Activities processed by Hotel Management System include calculation of restaurant bills, adding laundry bills, and the total cost. This diagram shows the "include" and "extend" relationships for depicting dependencies of these activities.

Student’s management system

The Working system:

- The following student management system manages all the work concerning students without having the need to log in, serving as an overall administrative tool efficiently handling different tasks related to students. It gives the administration the ability to add, view, search, and remove student records.
- Within the Add Students section, there is a pop-up that will appear for adding the name and ID of the student. Further, this shall be saved into the system for proper records. The View Student section opens up the list of all the records saved in it and presents the total overview, names, and IDs for a better preview.
- The search feature helps the user to get a specific student without wasting much time. Just enter the ID of the student, and it will look whether the record exists, displays all details of that student. Remove: It will delete the information of a student if its ID is provided. That would help remove outdated records or records that are no longer required.
- This system is quite important for keeping students' information safe, recording the total number of students, and ease of access to their information. It enhances administrative works a lot, hence making it easier to handle and visualize the records of students by school or university staff while managing to keep everything organized and accessible.

THE CODE

The List:

- First of all, I opened a class named student with two attributes the name and the ID, it means when students object is created it takes the name in the id as inputs and assigns them to the object.
- Moving to the next class I opened and named it students management system, and this is the main part where it stores all the students records in an array list called students. Since according to Lawrence University (n.d.), arraylist is a dynamic data structure in java that makes it easier to work with collections than arrays. and I also used a scanner for the input. Next, I made a menu that gives the user choices to choose to add, view, search and remove options and to make it ask repeatedly I made a loop the do-while loop since according to javapoint (n.d.), “The java do-while loop is used to iterate a part of the program repeatedly, until the specified condition is true.” This is where I made 5 options if the user choose 1 means add, 2 means view, 3 means search, 4 means remove and 5 means exit. As the system will keep asking until the user type 5 and exit.

```java
import java.util.ArrayList;
import java.util.Scanner;

// Represent a student with name and ID
class Student {
    String name;
    int id;

    Student(String name, int id) {
        this.name = name;
        this.id = id;
    }
}

// Stor students record and scan user's input
public class Studentsmanagementsystem {
    private static ArrayList<Student> students = new ArrayList<>();
    private static Scanner scanner = new Scanner(System.in);

    Run | Debug
    public static void main(String[] args) {
        int choice;
        // Show Options and print the choice
        do {
            System.out.println(x:"1. Add Student");
            System.out.println(x:"2. View Students");
            System.out.println(x:"3. Search Student");
            System.out.println(x:"4. Remove Student");
            System.out.println(x:"5. Exit");
            System.out.print(s:"Enter your choice: ");
            choice = scanner.nextInt();
            scanner.nextLine();

        // Perform actions
        switch (choice) {
            case 1:addStudent();break;
            case 2:viewStudents();break;
            case 3:searchStudent();break;
            case 4:removeStudent();break;
            case 5:System.out.println(x:"Exiting...");break;
            default:
                System.out.println(x:"Invalid choice. Please try again.");
        }
    } while (choice != 5); // a loop keep asking until press exit.
}
```

Add student:

- In this section, I developed a method called addStudent() that allows the user to add a new student in the collection 'students'. This method asks the user to input the name and ID of the student when called using the scanner. The two inputs are stored in the variables 'name' and 'id', respectively. It then prints out a confirmation message that a student has been added to the collection after the data is collected. A student's information is hence stored and duly acknowledged.

```
// Add new student name and id.
private static void addStudent() {
    System.out.print(s:"Enter student name: ");
    String name = scanner.nextLine();
    System.out.print(s:"Enter student ID: ");
    int id = scanner.nextInt();
    students.add(new Student(name, id));
    System.out.println(x:"Student added successfully.");
}
```

View student:

- This part relies on the ViewStudents() methos where it allows to view all students stored, by first checking if the collection is empty with students.isEmpty(). If there was no student, it appears a message saying that no students found. Otherwise, it appears a list of the students available. For each student it prints the name, and the id of students found.

```
// View students list
private static void viewStudents() {
    if (students.isEmpty()) {
        System.out.println(x:"No students found.");
    } else {
        System.out.println(x:"List of Students:");
        for (Student student : students) {
            System.out.println("Name: " + student.name + ", ID: " + student.id);
        }
    }
}
```

Search student:

- This searchStudent() method allows the user to look for a student by their ID, with first put the input in the scanner means write the id of the student. And we have a Boolean variable that initialized with false to track if the student is located. The methos works with for-each loop to check if the input matches the stored data, if a match was found it print student found with the name and the ID and set found to true then break out the loop. Otherwise, if no student was found it going to print student not found. This method simplifies tracking individual student records by their unique ID.

```java
// Search for students with ID
private static void searchStudent() {
    System.out.print(s:"Enter student ID to search: ");
    int id = scanner.nextInt();
    boolean found = false;
    for (Student student : students) {
        if (student.id == id) {
            System.out.println("Student found: Name: " + student.name + ", ID: " + student.id);
            found = true;
            break;
        }
    }
    if (!found) {
        System.out.println(x:"Student not found.");
    }
```

Remove student:

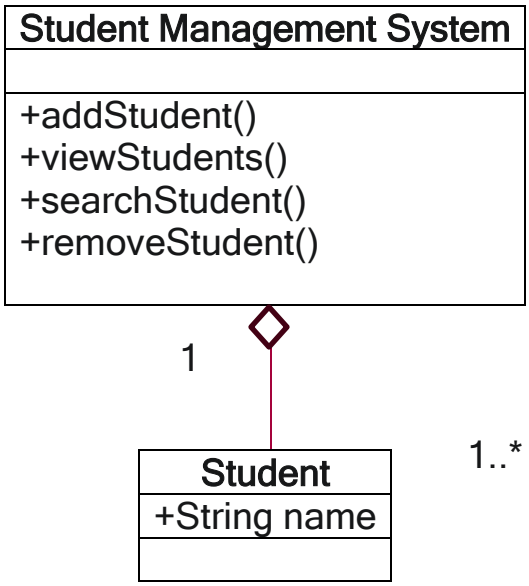
- This part of the code works with removeStudent() method which enable the user to remove a student from the list by typing their unique id. The methos starts by asking the id of the student that should be removed as the user should write an input it the scanner.
- And we have a Boolean initialized false to track if the removal was successful.
- This method works in a super easy way where the system compares the input with the id available if a match was found then the student will be removed from the list using the remove() method and print a message that says student was removed successfully. Otherwise, if no matching found, the method will print no student found. This method provides a clear

handling of the removal operation with feedback for both results.

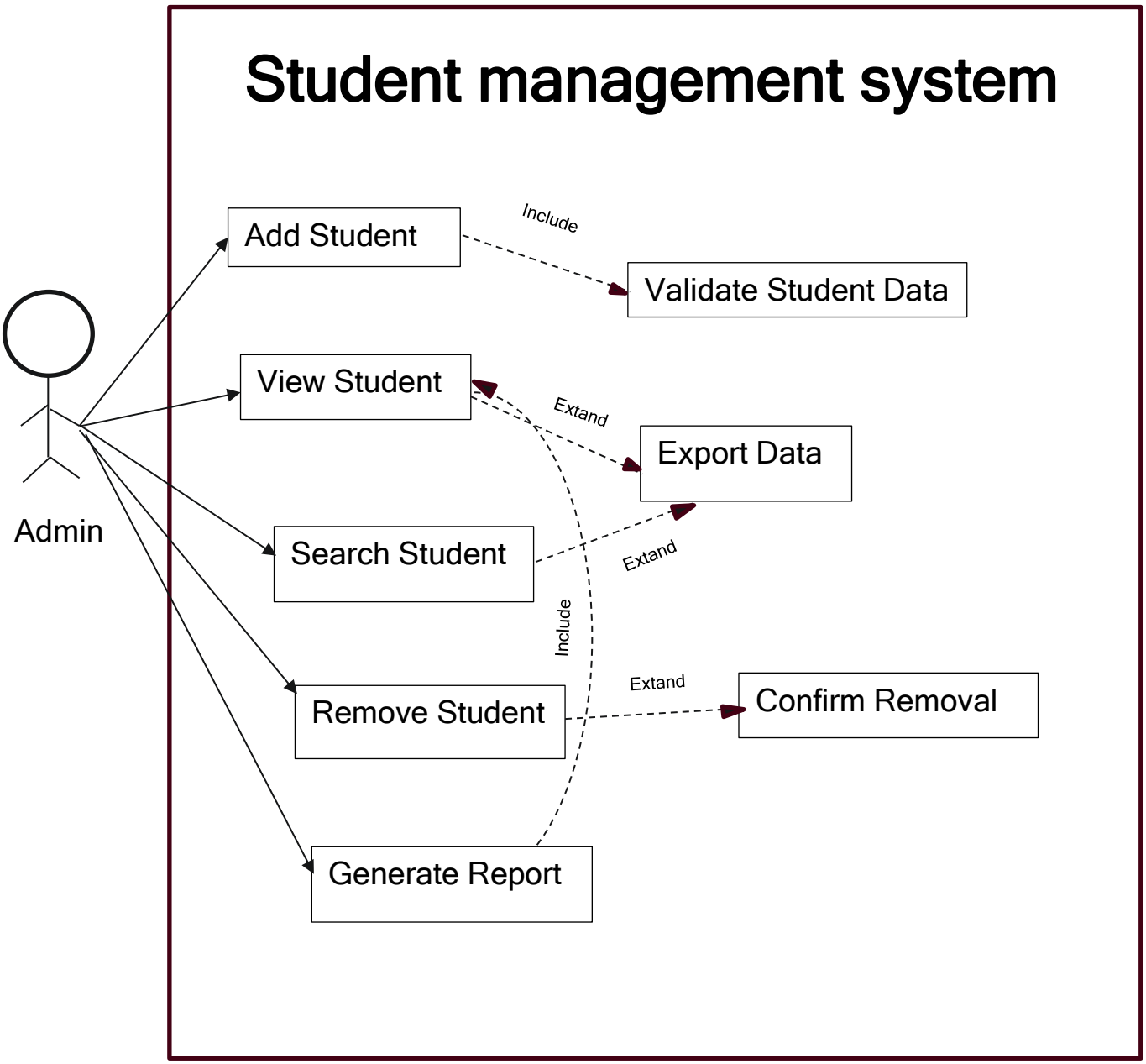
```
// Remove student from list with ID
private static void removeStudent() {
    System.out.print(s:"Enter student ID to remove: ");
    int id = scanner.nextInt();
    boolean removed = false;
    for (int i = 0; i < students.size(); i++) {
        if (students.get(i).id == id) {
            students.remove(i);
            System.out.println(x:"Student removed successfully.");
            removed = true;
            break;
        }
    }
    if (!removed) {
        System.out.println(x:"Student not found.");
    }
}
```

UML MODELING

- Any system requires some representative visual structure which may assist in understanding the requirement of understanding and analyzing a system. This shall always be developed through a UML diagram, giving clear and comprehensive visuals of the components constituting the whole system and relationships therein. Below is a UML for a student's management system listing important elements to show their interaction in the best possible manner to achieve the objects of the said system:



USE CASE



This is a use case diagram that shows a student management system. This includes an interaction of an Admin and the System, which grants permission for an Admin to add a student. Thus, the sequence further consists of valid students' data removal, followed by the confirmation of such removals-extend. View Student, search any student, which again extends to export data and report

generation. Dependencies among different various actions "include" and "extend" relations to highlight relations among connected workflow sequences.

CONCLUSION

- This report has looked at two key systems: the Hotel Management System and the Student Management System. It covered their design, features, and how they are set up. Both systems were created using Java, a strong and flexible programming language that can handle real-world needs and grow as needed.
- This hotel management system helps manage customer information, room booking and services such as restaurant, laundry and games. It exposes all the inefficiencies and error-proneness of manual hotel operations. It allows staff to track expenses throughout the period and produce summaries of customer activity to facilitate data processing and improve customer experience. The specific parts of this system being room rent calculation; restaurant billing; laundry services; games bill computation all collectively form the complete solution. Control structures of Java, like loop and switch statements helped in getting this functionalities UML modeling also enabled a well-defined representation of the system architecture, confirming the match between design and implementation.
- While the Student Management System helps the educational institutes to get rid of administrative obstacles. The project is divided into many sub functions, the system being able to add, view, search and delete student's records. An effective way to store and structure the student information so that the staff can access the records quickly and safely. If you compare the hotel system, dealing with various operations, and student system primarily concerned with the continuation of records only, you will find both systems are essentially adjustable, but the hotel system is more complex in terms of latency and administrative costs, as the hotel must account for training time and access complexities, while the student system must concern itself only with a familiar experience. ArrayLists in Java, to elaborate on Java's data structures to dynamically store and manipulate student records, The do-

while loop allowed the user to continually interact with the system until they wished to quit from the system, improving ease of use and the UML diagram to visualize the structure in a clear way.

- These systems introduce the importance of Java in building applications. This project allows the management of several hotels whereas this system helps maintaining the record of students in an easier manner. Their use of Java's Scanner class to handle user input suggests a clear planning behind their code structures. A control statement and loop implementation, called logic, allows the systems to be interactive and logical. While the UML diagrams represented by UML to visualize software architecture are important to communicate these concepts in diagrams and making sure that these two systems work perfectly.
- Such systems highlight the importance of management tools in contemporary organizational workflows. Automation saves money, as it saves you time, reduces human error, and improves overall productivity. By using Hotel Management System, the hotel employee no need to do a lot of work. Likewise, the Student Management System helps to maintain organized records, allowing educational institutes to run smoother. Technology can be used to address our needs in many areas.
- Overall, the implementation of both the Hotel Management System and the Student Management System demonstrates the power of Java as a programming language and the significance of the system design in enabling organizations. Not only does this new system led to greater efficiency but it also sets an example of how great software design can lead to transformational change, making things quicker to execute, and far more accurate and manageable. Incorporating structured programming and UML modeling, these systems are prime examples to real-world problems in respective domains.

REFERENCES

Ogirima, S. and Omuya, O., 2019. Online computerized Hotel Management System. Available at: https://www.semanticscholar.org/paper/Online-computerized-Hotel-Management-System-Ogirima-Omuya/58fbacd0a3313e1c986bd510f2121d51bbb257b1?p2df [Accessed 10 January 2025].

Oracle Corporation, 2014. The Java™ Language Specification, Java SE 8 Edition. Available at: https://docs.oracle.com/javase/specs/jls/se8/html/jls-14.html [Accessed 10 January 2025].

Rebus Press. (n.d.). If-then-else statements. In Programming Fundamentals. Retrieved January 10, 2025, from https://press.rebus.community/programmingfundamentals/chapter/if-then-else/?

Booch, G., Rumbaugh, J., & Jacobson, I., 1999. The Unified Modeling Language User Guide. Addison-Wesley.

Javatpoint, n.d. Java do-while loop. Available at: https://www.javatpoint.com/java-do-while-loop [Accessed 10 January 2025].

Lawrence University, n.d. Working with ArrayLists. Available at: https://www2.lawrence.edu/fast/GREGGJ/CMSC150/062ArrayLists/ArrayLists.html [Accessed 10 January 2025].